CEFET/RJ

R-BASICS

Eduardo Ogasawara
eogasawara@ieee.org
https://eic.cefet-rj.br/~eogasawara

Introduction to R

- R is a programming language and free software environment for statistical computing
  - Supported by the R Foundation for Statistical Computing
- Created by Ross Ihaka and Robert Gentleman at Auckland University, New Zealand
- R was derived by S (Bell Laboratories - AT&T)
- R is a language broadly used by statisticians, data miners, and data scientists
- Current version 4.3

R Console

RGui (64-bit) - [R Console]

Arquivo Editar Visualizar Misc Pacotes Janelas Ajuda

```
R version 4.2.1 (2022-06-23 ucrt) -- "Funny-Looking Kid"
Copyright (C) 2022 The R Foundation for Statistical Computing
Platform: x86_64-w64-mingw32/x64 (64-bit)

R é um software livre e vem sem GARANTIA ALGUMA.
Você pode redistribuí-lo sob certas circunstâncias.
Digite 'license()' ou 'licence()' para detalhes de distribuição.

R é um projeto colaborativo com muitos contribuidores.
Digite 'contributors()' para obter mais informações e
'citation()' para saber como citar o R ou pacotes do R em publicações.

Digite 'demo()' para demonstrações, 'help()' para o sistema on-line de ajuda,
ou 'help.start()' para abrir o sistema de ajuda em HTML no seu navegador.
Digite 'q()' para sair do R.

>
```

R Console (Basic operations)

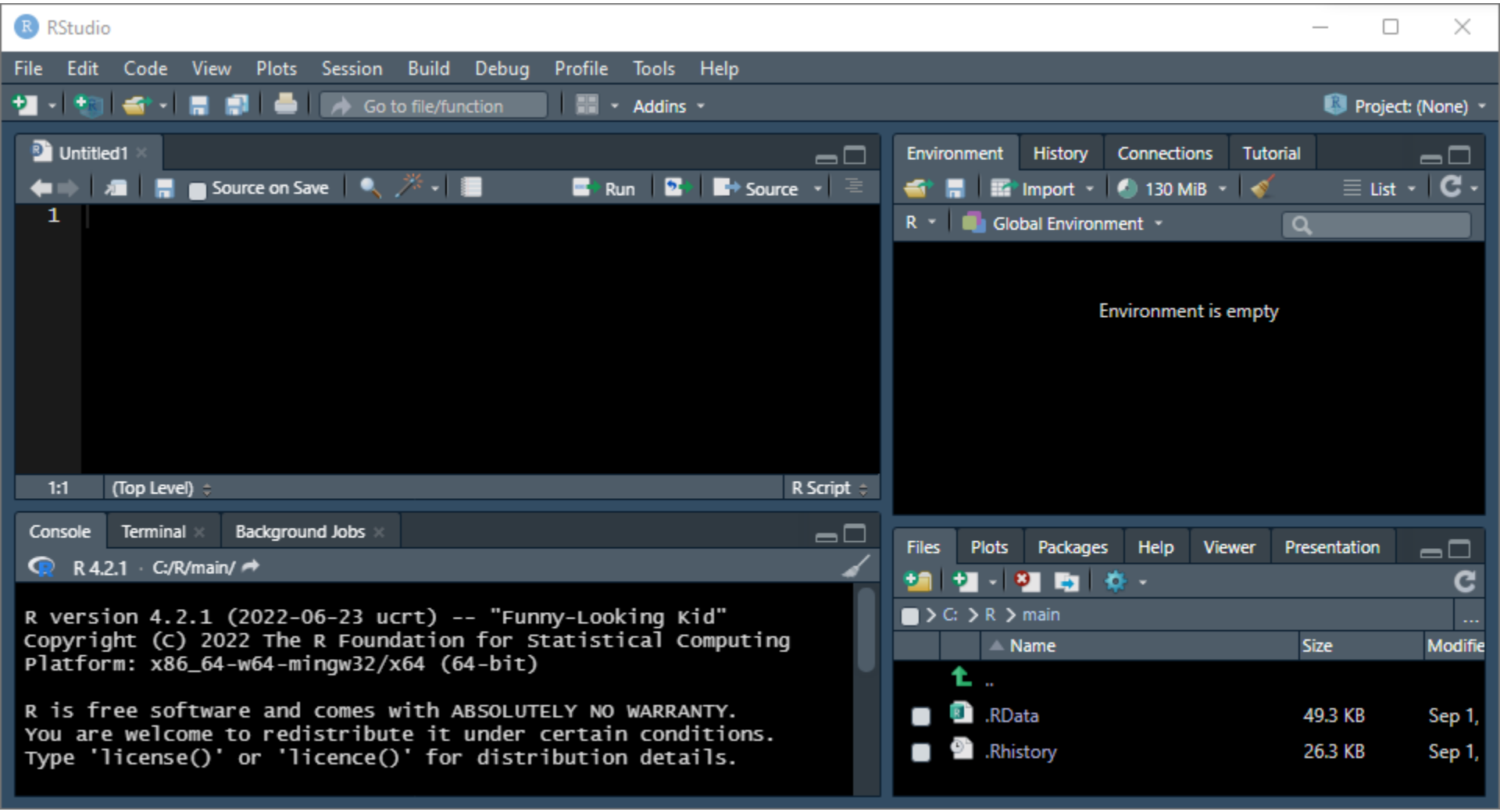
RGui (64-bit) - [R Console]

Arquivo Editar Visualizar Misc Pacotes Janelas Ajuda

```
> 1 + 2
[1] 3
> 1:10
 [1]  1  2  3  4  5  6  7  8  9 10
> a <- 1
> a
[1] 1
> a + 2
[1] 3
> dado <- 1:6
> dado
[1] 1 2 3 4 5 6
> dado[1]
[1] 1
> dado[6]
[1] 6
> dado - 1
[1] 0 1 2 3 4 5
>
```

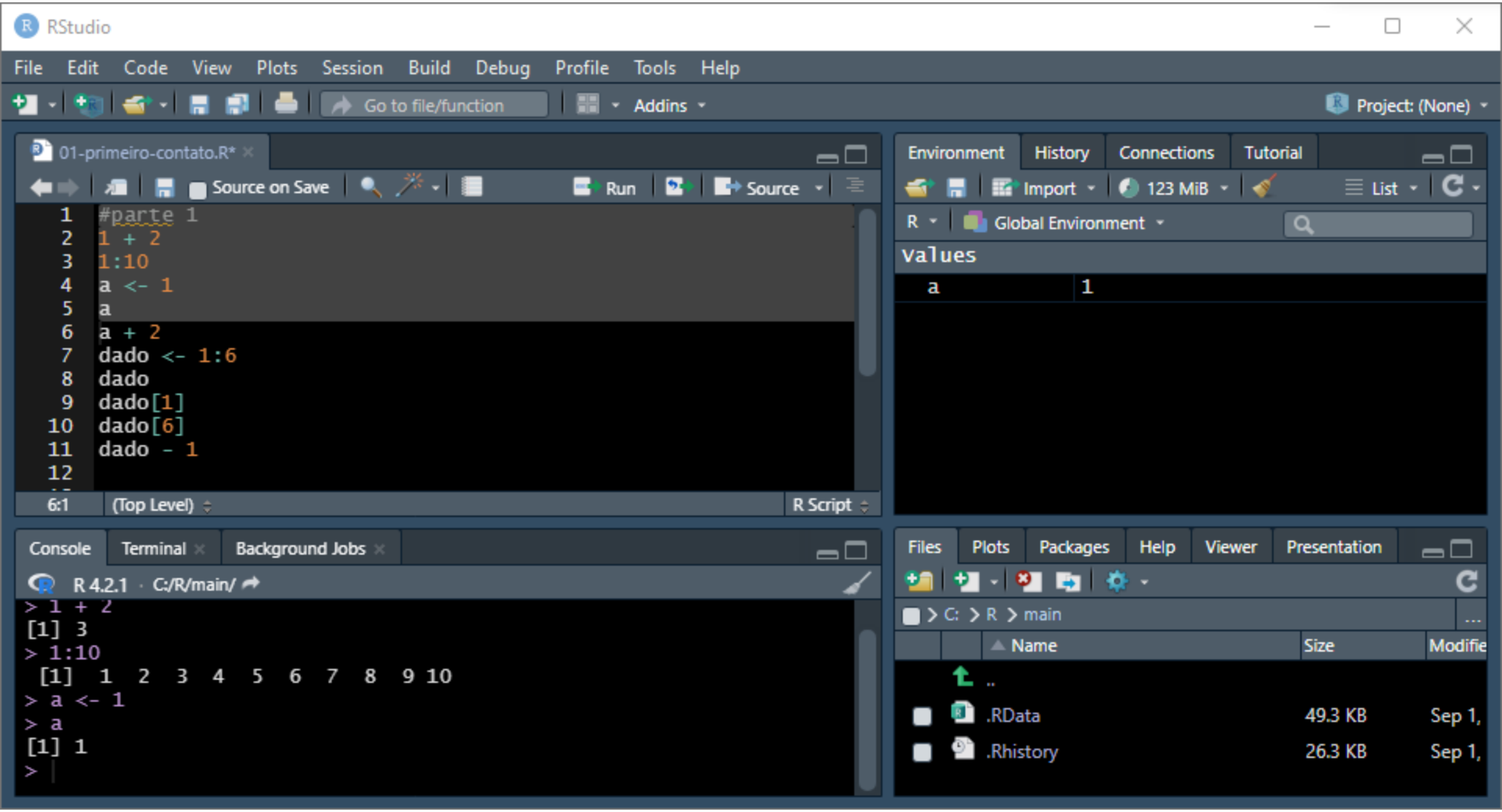
https://rstudio-education.github.io/hopr/basics.html

R Studio



Significant advantages: IDE with data visualization, debugging, profiling
http://www.rstudio.com

Run a block of code



Execução de trechos marcados no código (botão run)
Veja a janela de variáveis do ambiente

Run entire source file

RStudio

File Edit Code View Plots Session Build Debug Profile Tools Help

Go to file/function | Addins | Project: (None)

01-primeiro-contato.R

Source on Save | Run | Source

```
1  #parte 1
2  1 + 2
3  1:10
4  a <- 1
5  a
6  a + 2
7  dado <- 1:6
8  dado
9  dado[1]
10 dado[6]
11 dado - 1
12
```

12:1 (Top Level) R Script

Environment History Connections Tutorial

Import | 123 MiB | List

R | Global Environment

values

a	1
dado	int [1:6] 1 2 3 4 5 6

Console Terminal Background Jobs

R 4.2.1 · C:/R/main/

```
> source("C:/Users/eduar/OneDrive/Aulas/R/01-primeiro-contato.R")
>
```

Files Plots Packages Help Viewer Presentation

C: > R > main

Name	Size	Modifie
..		
.RData	49.3 KB	Sep 1,
.Rhistory	26.3 KB	Sep 1,

Execução do código inteiro (botão source)

R Packages

- Packages are collections of functions made available as libraries
  - Published in the CRAN Repository
    - Quality control
    - good documentation
  - Uploaded from a GitHub repository
    - Versions under development
- Currently have more than 20400 packages
  - https://cran.r-project.org/
- R has a very active community
  - Several researchers, professors, programmers, and statisticians
- DAL Packages
  - https://cran.r-project.org/web/packages/daltoolbox/index.html
  - https://cran.r-project.org/web/packages/TSPred/index.html
  - https://cran.rstudio.com/web/packages/gstsm/index.html
  - https://cran.r-project.org/web/packages/STMotif/index.html
  - https://cran.r-project.org/web/packages/harbinger/index.html
  - https://cran.r-project.org/web/packages/tspredit/index.html

https://www.r-project.org

R Package installation

RStudio

File Edit Code View Plots Session Build Debug Profile Tools Help

Go to file/function | Addins | Project: (None)

02-Pacotes-Ajuda.R*

Source on Save | Run | Source

```
#funções gráficas qplot
?qplot

library(ggplot2)

?qplot

install.packages("ggplot2")
```

8:1 (Top Level) R Script

Console Terminal Background Jobs

R 4.2.1 · C:/R/main/

```
Type 'demo()' for some demos, 'help()' for on-line help, or
'help.start()' for an HTML browser interface to help.
Type 'q()' to quit R.

[Workspace loaded from C:/R/main/.RData]

> ?qplot
No documentation for 'qplot' in specified packages and libraries:
you could try '??qplot'
> library(ggplot2)
> ?qplot
>
```

Environment History Connections Tutorial

Import | 132 MiB | List

R | Global Environment

Environment is empty

Files Plots Packages Help Viewer Presentation

R: Quick plot | Find in Topic

qplot {ggplot2} R Documentation

Quick plot

Description

Leave the install.packages commented to avoid running the installation several times

Using ggplot2 package

RStudio

File Edit Code View Plots Session Build Debug Profile Tools Help

Go to file/function | Addins | Project: (None)

02-Pacotes-Ajuda.R

Source on Save | Run | Source

```r
x <- c(-1, -0.8, -0.6, -0.4, -0.2, 0, 0.2, 0.4, 0.6, 0.8, 1)
x

y <- x^3
y

qplot(x, y)
```

1:1 (Top Level) R Script

Environment History Connections Tutorial

Import 178 MiB List

R Global Environment

values

x	num [1:11] -1 -0.8 -0.6 -0.4 ...
y	num [1:11] -1 -0.512 -0.216 -...

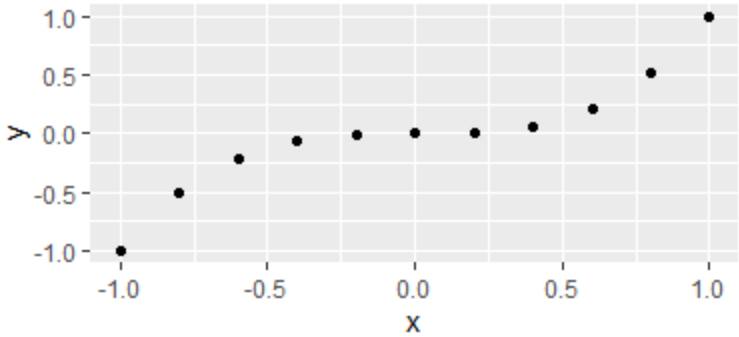
Console Terminal Background Jobs

R 4.2.1 · C:/R/main/

```
> x <- c(-1, -0.8, -0.6, -0.4, -0.2, 0, 0.2, 0.4, 0.6, 0.8, 1)
> x
 [1] -1.0 -0.8 -0.6 -0.4 -0.2  0.0  0.2  0.4  0.6  0.8  1.0
>
> y <- x^3
> y
 [1] -1.000 -0.512 -0.216 -0.064 -0.008  0.000  0.008  0.064
 [9]  0.216  0.512  1.000
>
> qplot(x, y)
>
```

Files Plots Packages Help Viewer Presentation

Zoom Export



Chamadas com parâmetros explícitos não precisa seguir a ordem os argumentos

Variable definitions

variable definitions

```
weight <- 60
height = 1.75
subject <- "A"
healthy <- TRUE
```

variable evaluation

```
weight
```

60

A string is a sequence of characters.
It is enclosed inside single quotes('This is a string') or inside the double quotes("This is also a string").
It is internally represented in double quotes.

Variable checking and conversion

variable type checking

```
is.numeric(weight) # variable
is.double(weight)
is.integer(weight)
is.character(subject)
```

TRUE

TRUE

FALSE

TRUE

variable conversion

```
weight <- as.integer(weight)
is.integer(weight)
```

TRUE

Family of functions is.* evaluates data type: Type “is.” and press space to see all them.
Family of functions as.* converts data types: Type “as.” and press space to see all them.

Formulas and String formatting

Formulas

```
#Body mass index (BMI)
bmi <- weight/height^2
bmi
```

19.5918367346939

String formatting

```
message <- sprintf("%.1f", bmi)
print(message)
```

```
[1] "19.6"
```

Function sprintf is like Java, C++, and C.

Vectors

Vector definition

```
weight <- c(60, 72, 57, 90, 95, 72)
height <- c(1.75, 1.80, 1.65, 1.90, 1.74, 1.91)
subject <- c("A", "B", "C", "D", "E", "F")
```

Vector evaluation

```
weight
height
subject
```

60 · 72 · 57 · 90 · 95 · 72

1.75 · 1.8 · 1.65 · 1.9 · 1.74 · 1.91

'A' · 'B' · 'C' · 'D' · 'E' · 'F'

Creating a vector with a particular size

```
vec <- rep(0, 10)
vec
```

0 · 0 · 0 · 0 · 0 · 0 · 0 · 0 · 0 · 0

Function c creates a vector from static values.
Function rep creates a vector of a particular size.

Vectors

Vector length

```
length(weight)
```

6

Vector indexes

```
weight[1]
weight[length(weight)]
```

60

72

Function length presents the size of a vector.
A vector starts at index one and ends at its length.

Iterations

for loop

```
bmi <- 0
for (i in 1:length(weight)) {
  bmi[i] <- weight[i]/height[i]^2
}
```

```
bmi
```

19.5918367346939 · 22.2222222222222 · 20.9366391184573 · 24.9307479224377 · 31.3779891663364 · 19.7363010882377

while loop

```
i <- 1
while (i <= length(weight)) {
  bmi[i] <- weight[i]/height[i]^2
  i <- i + 1
}
```

```
bmi
```

19.5918367346939 · 22.2222222222222 · 20.9366391184573 · 24.9307479224377 · 31.3779891663364 · 19.7363010882377

For loops: defined such that variable iterates over all elements of a vector.
While loops: are executed while a condition is satisfied, it is expected that variables for condition evaluation are changed inside the loop.

Removing variables

remove variable

```
rm(bmi)
exists("bmi")
```

FALSE

The function rm removes the variable and free memory space.
The function exists checks to see if a variable with a specified name exists.

Vectors in R

Manipulating vectors efficiently

```
bmi <- weight/height^2
bmi
```

19.5918367346939 · 22.2222222222222 · 20.9366391184573 · 24.9307479224377 · 31.3779891663364 · 19.7363010882377

Vectors do not need loops for most cases.

Creating functions

Function definition

```
compute_bmi <- function(weight, height) {
  bmi <- weight/height^2
  return(bmi)
}
```

Using with scalar

```
bmi <- compute_bmi(60, 1.75)
bmi
```

19.5918367346939

Using with vectors

```
bmi <- compute_bmi(weight, height)
bmi
```

19.5918367346939 · 22.2222222222222 · 20.9366391184573 · 24.9307479224377 · 31.3779891663364 · 19.7363010882377

Functions are defined with parameters and code assigned to a function name.
Functions should return a value.

Example

- Knowing that mean is computed as
  - $\mu(X) = \frac{\sum_{i=1}^{n} x_i}{n}$
- Create a function to compute the mean of a vector

Mean is the sum of observations divided by the number of observations.

Example: function to compute the average

average function

```
average <- function(vec) {
    s <- 0
    n <- length(vec)
    for (x in vec) {
      s <- s + x
    }
    return(s/n)
}
```

```
avg_bmi <- average(bmi)
avg_bmi
```

23.1326227087309

Average computed using a for loop: similar to most programming languages.

Example: function to compute the average (improved version)

average function: improved version

```
average <- function(vec) {
    s <- sum(vec)
    n <- length(vec)
    return(s/n)
}
```

```
avg_bmi <- average(bmi)
avg_bmi
```

23.1326227087309

Average computed using functions applied to the entire vectors.

Example: function to compute the average (major statistical functions exists)

average function: major statistical functions already exists

```
average <- function(vec) {
    return(mean(vec))
}
```

```
avg_bmi <- average(bmi)
avg_bmi
```

23.1326227087309

Since R was initially from the statistical community, it contains hundreds of statistical methods.

Handling NA
evaluation with NA leads to NA

NA definition

```
x <- c(10, NA, 13)
x
```

10 · <NA> · 13

Evaluation with NA leads to NA

```
average(x)
```

<NA>

Addressing NA

```
average <- function(vec) {
    return(mean(vec, na.rm=TRUE))
}
```

```
average(x)
```

11.5

Any operation with NA produces NA results.
The na.rm=TRUE parameter removes NA observations.

Part II

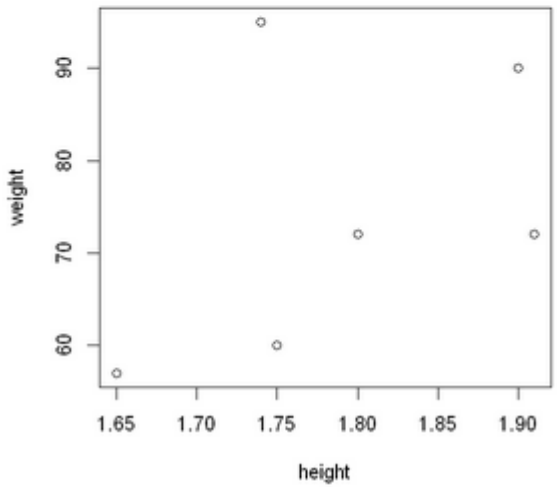
Plotting charts

Setup variables

```
weight <- c(60, 72, 57, 90, 95, 72)
height <- c(1.75, 1.80, 1.65, 1.90, 1.74, 1.91)
subject <- c("A", "B", "C", "D", "E", "F")
```

Basic plot

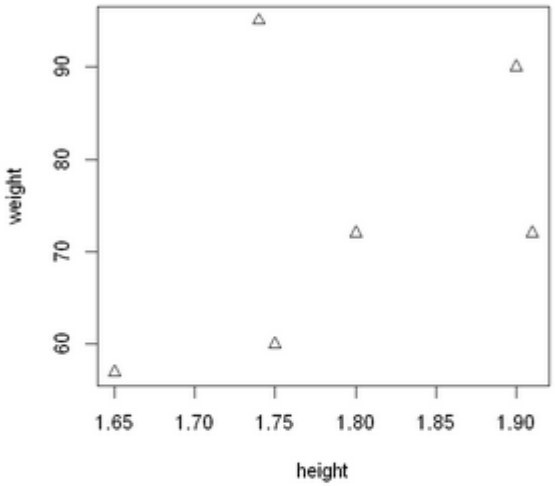
```
#options(repr.plot.width=5, repr.plot.height=5)
plot(height, weight)
```



The plot function is the basic plot function in R. The first parameter is the x-axis observations. In contrast, the second one is the y-axis observations.

Default parameters

```
plot(height, weight, pch=2)
```




```
args(plot.default)
```

```
function (x, y = NULL, type = "p", xlim = NULL, ylim = NULL,
    log = "", main = NULL, sub = NULL, xlab = NULL, ylab = NULL,
    ann = par("ann"), axes = TRUE, frame.plot = axes, panel.first = NULL,
    panel.last = NULL, asp = NA, xgap.axis = NA, ygap.axis = NA,
    ...)
NULL
```

Functions in R contain many default parameters.
The parameters of functions can be discovered using the args function.

Help for functions

```
?plot
```

plot.default {graphics} R Documentation

The Default Scatterplot Function

Description

Draw a scatter plot with decorations such as axes and titles in the active graphics window.

Usage

```
## Default S3 method:
plot(x, y = NULL, type = "p",  xlim = NULL, ylim = NULL,
     log = "", main = NULL, sub = NULL, xlab = NULL, ylab = NULL,
     ann = par("ann"), axes = TRUE, frame.plot = axes,
     panel.first = NULL, panel.last = NULL, asp = NA,
     xgap.axis = NA, ygap.axis = NA,
     ...)
```

Arguments

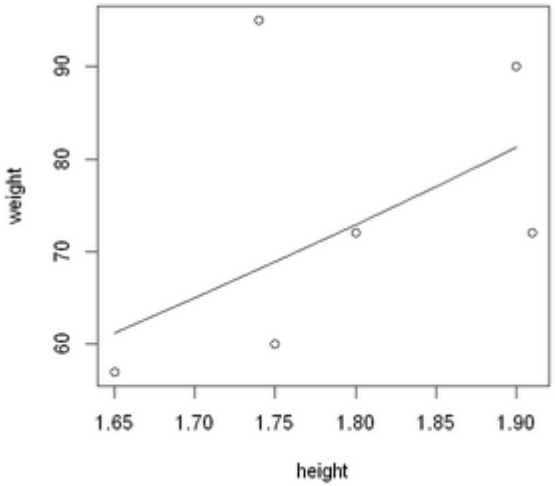
`x, y`	the `x` and `y` arguments provide the x and y coordinates for the plot. Any reasonable way of defining the coordinates is acceptable. See the function `xy.coords` for details. If supplied separately, they must be of the same length.
`type`	1-character string giving the type of plot desired. The following values are possible, for details, see

All functions in R contain help documentation with example code available.

Plotting charts
Canvas is still active

Adding a line

```
plot(height, weight)
hh = c(1.65, 1.70, 1.75, 1.80, 1.85, 1.90)
lines(hh, 22.5 * hh^2)
```



Canvas is active until the following plot. Lines add added the current scatter plot.

Factors

- Factors are variables in R that refer to categorical data
- Factors in R are stored as a vector of integer values with a corresponding set of character values to use when the factor is displayed
- Both numeric and character variables can be made into factors, but a factor's levels are always character values

The factors can be either sorted (ordinal) or unsorted (categorical).

Factors

Factors

```
pain <- c(0,3,2,2,1)
fpain <- factor(pain,levels=0:3, ordered=TRUE)
fpain
```

0 · 3 · 2 · 2 · 1

▾ **Levels**:

'0' · '1' · '2' · '3'

Labeling categories

```
levels(fpain) <- c("none","mild","medium","severe")
fpain
```

none · severe · medium · medium · mild

▾ **Levels**:

'none' · 'mild' · 'medium' · 'severe'

Example of ordered factor indicating the level of pain for patients.

Example

- Consider the height variable
  - Persons lower than 1.5 m are small
  - Persons greater than 1.9 m are tall
  - Persons in between are medium
  - Convert the height variable into a factor with small, medium, tall

Double to factors

Convert double to factor

```
lev <- rep("", length(height))

for (i in 1:length(height)) {
  if (height[i] < 1.7)
    lev[i] <- "short"
  else if (height[i] < 1.9)
    lev[i] <- "medium"
  else
    lev[i] <- "tall"
}
lev <- as.factor(lev)
lev
```

medium · medium · short · tall · medium · tall

▾ **Levels**:

'medium' · 'short' · 'tall'

It is an intuitive way of transforming a double vector into a factor.

Double to factors (Efficient way)

Using cut function

```
lev <- cut(height, breaks=c(0, 1.7, 1.9, .Machine$double.xmax), ordered=TRUE)
lev
levels(lev) <- c("short", "medium", "tall")
lev
```

(1.7,1.9] · (1.7,1.9] · (0,1.7] · (1.7,1.9] · (1.7,1.9] · (1.9,1.8e+308]

▾ **Levels**:

'(0,1.7]' · '(1.7,1.9]' · '(1.9,1.8e+308]'

medium · medium · short · medium · medium · tall

▾ **Levels**:

'short' · 'medium' · 'tall'

Function cut was designed for such a conversion.

Matrix
(Matrix is a two-dimensional data structure in R programming)

```
x <- 1:9
x
```

1 · 2 · 3 · 4 · 5 · 6 · 7 · 8 · 9

vector to matrix

```
dim(x) <- c(3,3)
x
```

A matrix: 3 × 3 of type int

1	4	7
2	5	8
3	6	9

Dimensions of a vector can be changed to transform it into a matrix.

Matrix
(accessing values)

```
x
```

A matrix: 3
× 3 of type
int

1	4	7
2	5	8
3	6	9

```
for (i in 1:nrow(x))
  for (j in 1:ncol(x))
    print(x[i,j])
```

```
[1] 1
[1] 4
[1] 7
[1] 2
[1] 5
[1] 8
[1] 3
[1] 6
[1] 9
```

Elements of a matrix are accessed from row and column.
Function nrow indicates the number of rows in a matrix. Function ncol indicates the number of columns of a matrix.

Matrix
(scalar multiplication)

multiplying values by 3

```
y <- x
for (i in 1:nrow(y))
    for (j in 1:ncol(y))
        y[i,j] <- 3 * y[i, j]

y
```

A matrix: 3 × 3 of type dbl

3	12	21
6	15	24
9	18	27

```
y <- 3*x
y
```

A matrix: 3 × 3 of type dbl

3	12	21
6	15	24
9	18	27

Elements can be changed one by one or at once using scalar operations.

Matrix
(fill by row - transpose)

Vector to matrix by row

```
x <- matrix(1:9,nrow=3,byrow=T)
x
```

A matrix: 3
× 3 of type
int

1	2	3
4	5	6
7	8	9

Transpose matrix

```
x <- t(x)
x
```

A matrix: 3
× 3 of type
int

1	4	7
2	5	8
3	6	9

The conversion of a vector into a matrix can be defined in a row-based filling.
A matrix can be transposed using function t.

Lists

- Lists are the R objects which contain elements of different types, such as numbers, strings, vectors, matrix, data frame, and another list inside it
- A list can also contain a matrix or a function as its elements
- A list is created using the list() function
- List manipulation
  - Slicing a list []
  - Accessing an element inside a list [[]]

Creating lists

Lists are used to work with "objects"

```
a <- c(5260,5470,5640,6180,6390,6515,6805,7515,7515,8230,8770)
b <- c(3910,4220,3885,5160,5645,4680,5265,5975,6790,6900,7335)

mybag <- list(a, b, 0, "a")
mybag
```

1. 5260 · 5470 · 5640 · 6180 · 6390 · 6515 · 6805 · 7515 · 7515 · 8230 · 8770
2. 3910 · 4220 · 3885 · 5160 · 5645 · 4680 · 5265 · 5975 · 6790 · 6900 · 7335
3. 0
4. 'a'

adding an element into a list

```
n <- length(mybag)
mybag[[n+1]] <- "b"
mybag
```

1. 5260 · 5470 · 5640 · 6180 · 6390 · 6515 · 6805 · 7515 · 7515 · 8230 · 8770
2. 3910 · 4220 · 3885 · 5160 · 5645 · 4680 · 5265 · 5975 · 6790 · 6900 · 7335
3. 0
4. 'a'
5. 'b'

Lists can contain anything, including lists. To add a new item, use an empty position.

List slicing

Slicing a list

```
slice <- mybag[1]
slice
is.list(slice)
```

1. 5260 · 5470 · 5640 · 6180 · 6390 · 6515 · 6805 · 7515 · 7515 · 8230 · 8770

TRUE

```
slice <- mybag[c(1,3)]
slice
is.list(slice)
```

1. 5260 · 5470 · 5640 · 6180 · 6390 · 6515 · 6805 · 7515 · 7515 · 8230 · 8770
2. 0

TRUE

```
#list is also a vector
is.vector(slice)
```

TRUE

Lists accessed by single brackets return sublists.

Lists: Member reference

Extracting an element from a list

```
h <- mybag[[1]]
h
```

5260 · 5470 · 5640 · 6180 · 6390 · 6515 · 6805 · 7515 · 7515 · 8230 · 8770

```
is.vector(h)
is.list(h)
```

TRUE

FALSE

To access the elements of a list, use double brackets.

Creating lists

Adding, accessing, and removing elements

```
mybag <- list(x=a, y=b, const=0, lit="a")
mybag
```

$x

5260 · 5470 · 5640 · 6180 · 6390 · 6515 · 6805 · 7515 · 7515 · 8230 · 8770

$y

3910 · 4220 · 3885 · 5160 · 5645 · 4680 · 5265 · 5975 · 6790 · 6900 · 7335

$const

0

$lit

'a'

```
mybag$c <- mybag$x - mybag$y
mybag[[4]] <- NULL
mybag$const <- NULL
mybag
```

$x

5260 · 5470 · 5640 · 6180 · 6390 · 6515 · 6805 · 7515 · 7515 · 8230 · 8770

$y

3910 · 4220 · 3885 · 5160 · 5645 · 4680 · 5265 · 5975 · 6790 · 6900 · 7335

$c

1350 · 1250 · 1755 · 1020 · 745 · 1835 · 1540 · 1540 · 725 · 1330 · 1435

Elements of lists can be named. It can be understood as an attribute of an object.
Operator $ is used to access the attribute of a list.

Part III

Data frames

- A data frame is a table where each column corresponds to attributes, and each row corresponds to a tuple (object)

Data frames (creation)

```
a <- c(5260,5470,5640,6180,6390,6515,6805,7515,7515,8230,8770)
b <- c(3910,4220,3885,5160,5645,4680,5265,5975,6790,6900,7335)
```

```
d <- data.frame(A=a, B=b)
head(d)
```

A data.frame: 6 × 2

	A	B
	<dbl>	<dbl>
1	5260	3910
2	5470	4220
3	5640	3885
4	6180	5160
5	6390	5645
6	6515	4680

Data frames can be built from vectors of the same size.

Data frames (adding columns)

```
d$C <- d$A + d$B
head(d)
```

A data.frame: 6 × 3

	A	B	C
	<dbl>	<dbl>	<dbl>
1	5260	3910	9170
2	5470	4220	9690
3	5640	3885	9525
4	6180	5160	11340
5	6390	5645	12035
6	6515	4680	11195

Each column has a name. Novel columns can be added by assigning a value to them. Operator $ is used to access the column of a table.

Data frames (removing columns)

```
d$A <- NULL
head(d)
```

A data.frame: 6 × 2

	B	C
	<dbl>	<dbl>
1	3910	9170
2	4220	9690
3	3885	9525
4	5160	11340
5	5645	12035
6	4680	11195

Columns of a data frame can be removed, assigning null to it.

Reading a data frame from csv file

```
wine = read.table(
  "http://archive.ics.uci.edu/ml/machine-learning-databases/wine/wine.data",
  header = TRUE, sep = ",")
head(wine)
```

A data.frame: 6 × 14

	X1	X14.23	X1.71	X2.43	X15.6	X127	X2.8	X3.06	X.28	X2.29	X5.64	X1.04	X3.92	X10
	<int>	<dbl>	<dbl>	<dbl>	<dbl>	<int>	<dbl>	<dbl>	<dbl>	<dbl>	<dbl>	<dbl>	<dbl>	<in
1	1	13.20	1.78	2.14	11.2	100	2.65	2.76	0.26	1.28	4.38	1.05	3.40	10
2	1	13.16	2.36	2.67	18.6	101	2.80	3.24	0.30	2.81	5.68	1.03	3.17	11
3	1	14.37	1.95	2.50	16.8	113	3.85	3.49	0.24	2.18	7.80	0.86	3.45	14
4	1	13.24	2.59	2.87	21.0	118	2.80	2.69	0.39	1.82	4.32	1.04	2.93	7
5	1	14.20	1.76	2.45	15.2	112	3.27	3.39	0.34	1.97	6.75	1.05	2.85	14
6	1	14.39	1.87	2.45	14.6	96	2.50	2.52	0.30	1.98	5.25	1.02	3.58	12

CSV files can be read locally or remotely using read.table function.

Saving and loading Rdata files

Saving RData file

```
# in Linux the default is not to compress
save(wine, file="wine.RData", compress=TRUE)
```

Removing data frame from memory

```
rm(wine)
```

Load RData file

```
load("wine.RData")
head(wine, 3)
```

A data.frame: 3 × 14

	X1	X14.23	X1.71	X2.43	X15.6	X127	X2.8	X3.06	X.28	X2.29	X5.64	X1.04	X3.92	X10
	<int>	<dbl>	<dbl>	<dbl>	<dbl>	<int>	<dbl>	<dbl>	<dbl>	<dbl>	<dbl>	<dbl>	<dbl>	<in
1	1	13.20	1.78	2.14	11.2	100	2.65	2.76	0.26	1.28	4.38	1.05	3.40	10
2	1	13.16	2.36	2.67	18.6	101	2.80	3.24	0.30	2.81	5.68	1.03	3.17	11
3	1	14.37	1.95	2.50	16.8	113	3.85	3.49	0.24	2.18	7.80	0.86	3.45	14

R has some functions to save and load variables as a binary file.

Saving CSV file

Export data.frame into csv file

```
write.table(wine, file="wine.csv", row.names=FALSE, quote = FALSE, sep = ",")
```

```
X1,X14.23,X1.71,X2.43,X15.6,X127,X2.8,X3.06,X.28,X2.29,X5.64,X1.04,X3.92,X1065
1,13.2,1.78,2.14,11.2,100,2.65,2.76,0.26,1.28,4.38,1.05,3.4,1050
1,13.16,2.36,2.67,18.6,101,2.8,3.24,0.3,2.81,5.68,1.03,3.17,1185
1,14.37,1.95,2.5,16.8,113,3.85,3.49,0.24,2.18,7.8,0.86,3.45,1480
1,13.24,2.59,2.87,21,118,2.8,2.69,0.39,1.82,4.32,1.04,2.93,735
```

Function write.table can save a CSV file.

Filtering vectors

Filtering

```
a <- c(5260,5470,5640,6180,6390,6515,6805,7515,7515,8230,8770)
b <- c(3910,4220,3885,5160,5645,4680,5265,5975,6790,6900,7335)
```

```
# logical vector
i <- (a > 7000)
i
```

FALSE · FALSE · FALSE · FALSE · FALSE · FALSE · FALSE · TRUE · TRUE · TRUE · TRUE

```
a[i]
```

7515 · 7515 · 8230 · 8770

```
b[a < 6000 | a > 7000]
```

3910 · 4220 · 3885 · 5975 · 6790 · 6900 · 7335

```
b[6000 <= a & a <= 7000]
```

5160 · 5645 · 4680 · 5265

Conditional expressing returns logical vectors.
Logical vectors can be used to filter vectors and data frames.

Filtering data frames

filtering data frames

```
data <- data.frame(a=a, b=b)
data$c <- data$a - data$b
head(data, nrow(data))
```

A data.frame: 11 × 3

	a	b	c
	<dbl>	<dbl>	<dbl>
1	5260	3910	1350
2	5470	4220	1250
3	5640	3885	1755
4	6180	5160	1020
5	6390	5645	745
6	6515	4680	1835
7	6805	5265	1540
8	7515	5975	1540
9	7515	6790	725
10	8230	6900	1330
11	8770	7335	1435

```
head(data[data$a > 7000,])
```

A data.frame: 4 × 3

	a	b	c
	<dbl>	<dbl>	<dbl>
8	7515	5975	1540
9	7515	6790	725
10	8230	6900	1330
11	8770	7335	1435

Filtering data frames should be done for the row index of the data frame (before the comma inside the brackets).

Filtering and projecting data frames

filtering data frames

```
data <- data.frame(a=a, b=b)
data$c <- data$a - data$b
head(data, nrow(data))
```

A data.frame: 11 × 3

	a	b	c
	<dbl>	<dbl>	<dbl>
1	5260	3910	1350
2	5470	4220	1250
3	5640	3885	1755
4	6180	5160	1020
5	6390	5645	745
6	6515	4680	1835
7	6805	5265	1540
8	7515	5975	1540
9	7515	6790	725
10	8230	6900	1330
11	8770	7335	1435

```
head(data[data$a > 7000,c(1,2)])
```

A data.frame: 4 × 2

	a	b
	<dbl>	<dbl>
8	7515	5975
9	7515	6790
10	8230	6900
11	8770	7335

Constrains for the columns of a data frame (after the comma of brackets) are projections.

Performance issues with matrix versus data.frame

Benchmarking matrix and data.frames

```
lib <- require(pryr)
if (!lib)
    install.packages("pryr")
library(pryr)
```

```
rheight <- rnorm(100000, 1.8, sd=0.2)
rweight <- rnorm(100000, 72, sd=15)
```

Computing an entire column at once

```
start_time <- Sys.time()
hw <- data.frame(height=rheight, weight=rweight)
hw$bmi <- hw$weight/hw$height^2
end_time <- Sys.time()
end_time - start_time
object_size(hw)
```

```
Time difference of 0.01570988 secs

2,400,984 B
```

rnorm(n, m, s) generates n random numbers following a normal distribution with mean m and standard deviation s
Package pryr is used to measure memory consumption for variables
Sys.time() is used to measure elapsed time

Performance issues with matrix versus data.frame

Processing element by element

```
start_time <- Sys.time()
hw <- data.frame(height=rheight, weight=rweight)
for (i in 1:nrow(hw)) {
  hw$bmi[i] <- hw$weight[i]/hw$height[i]^2
}
end_time <- Sys.time()
end_time - start_time
```

```
Time difference of 11.97988 secs
```

Convert the entire data.frame

```
start_time <- Sys.time()
hw <- data.frame(height=rheight, weight=rweight)
hw <- as.matrix(hw)
hw <- cbind(hw, 0)
for (i in 1:nrow(hw)) {
  hw[i,3] <- hw[i,2]/hw[i,1]^2
}
end_time <- Sys.time()
end_time - start_time
```

```
Time difference of 0.2502141 secs
```

Assigning values for each element in the data frame is time-consuming.
It is better to convert a data frame into a matrix and process it as a matrix.

Implicitly Loops – sapply, lapply

- lapply, sapply executes a function for each column
  - The first character defines the return type
    - l – list, s – simple (vector or matrix)
  - The second parameter is the function to invoke
  - Following parameters are passed to the invoked function
- apply is the generic function
  - The second parameter defines if it calls the function for each row (1) or each column (2)

Implicitly Loops – sapply, lapply

apply family

```
library(ISwR)
data(thuesen)
head(thuesen)
```

A data.frame: 6 × 2

	blood.glucose	short.velocity
	<dbl>	<dbl>
1	15.3	1.76
2	10.8	1.34
3	8.1	1.27
4	19.5	1.47
5	7.2	1.27
6	5.3	1.49

Implicitly Loops – sapply, lapply

```
lapply(thuesen, mean, na.rm=T)
```

$blood.glucose
10.3
$short.velocity
1.32565217391304

```
sapply(thuesen, mean, na.rm=T)
```

blood.glucose: 10.3 **short.velocity:** 1.32565217391304

lapply and sapply process each column invoking the function passed as second parameter.
All the following parameters are passed to the function.

Implicitly Loops – sapply, lapply

```
m <- as.matrix(thuesen)
apply(m, 1, min, na.rm=TRUE)
apply(m, 2, min, na.rm=TRUE)
```

1.76 · 1.34 · 1.27 · 1.47 · 1.27 · 1.49 · 1.31 · 1.09 · 1.18 · 1.22 · 1.25 · 1.19 ·
1.95 · 1.28 · 1.52 · 8.6 · 1.12 · 1.37 · 1.19 · 1.05 · 1.32 · 1.03 · 1.12 · 1.7

blood.glucose: 4.2 **short.velocity:** 1.03

The first parameter of apply is a matrix.
The second parameter of apply is the direction (1: row, 2: column)

Sort and order

sort and order

```
library(ISwR)
data(thuesen)
head(thuesen)
```

A data.frame: 6 × 2

	blood.glucose	short.velocity
	<dbl>	<dbl>
1	15.3	1.76
2	10.8	1.34
3	8.1	1.27
4	19.5	1.47
5	7.2	1.27
6	5.3	1.49

The thuesen dataset is available at ISwR package

Sort and order

```
sort(thuesen$blood.glucose)
```

4.2 · 4.9 · 5.2 · 5.3 · 6.7 · 6.7 · 7.2 · 7.5 · 8.1 · 8.6 · 8.8 · 9.3 · 9.5 · 10.3 · 10.8 · 11.1 · 12.2 · 12.5 · 13.3 · 15.1 · 15.3 · 16.1 · 19 · 19.5

```
order(thuesen$blood.glucose)
```

17 · 22 · 12 · 6 · 11 · 15 · 5 · 9 · 3 · 16 · 23 · 7 · 24 · 18 · 2 · 8 · 10 · 19 · 21 · 14 · 1 · 20 · 13 · 4

```
o <- order(thuesen$blood.glucose)
sorted <- thuesen[o,]
head(sorted)
```

A data.frame: 6 × 2

	blood.glucose	short.velocity
	<dbl>	<dbl>
17	4.2	1.12
22	4.9	1.03
12	5.2	1.19
6	5.3	1.49
11	6.7	1.25
15	6.7	1.52

The sort function orders a vector. The order function gets the index from the original vector that corresponds to the sorted vector. Passing an integer vector for rows in a data frame selects the rows according to the specified order.

Part IV

Creating an example

```r
weight <- c(60, 72, 57, 90, 95, 72)
height <- c(1.75, 1.80, 1.65, 1.90, 1.74, 1.91)
subject <- c("A", "B", "C", "D", "E", "F")
d <- data.frame(weight=weight, height=height, subject=subject)
head(d)
```

A data.frame: 6 × 3

	weight	height	subject
	<dbl>	<dbl>	<chr>
1	60	1.75	A
2	72	1.80	B
3	57	1.65	C
4	90	1.90	D
5	95	1.74	E
6	72	1.91	F

Creating an auxiliar table

```r
subject <- c("A", "B", "C", "D", "E", "F")
state <- c("RJ", "SP", "MG", "RJ", "SP", "MG")
ds <- data.frame(subject=subject, state=state)
head(d)
```

A data.frame: 6 × 3

	weight	height	subject
	<dbl>	<dbl>	<chr>
1	60	1.75	A
2	72	1.80	B
3	57	1.65	C
4	90	1.90	D
5	95	1.74	E
6	72	1.91	F

Joining tables

```
dsm <- merge(d, ds, by.x="subject", by.y="subject")
head(dsm)
```

A data.frame: 6 × 4

	subject	weight	height	state
	<chr>	<dbl>	<dbl>	<chr>
1	A	60	1.75	RJ
2	B	72	1.80	SP
3	C	57	1.65	MG
4	D	90	1.90	RJ
5	E	95	1.74	SP
6	F	72	1.91	MG

Data transformation

```
library(dplyr)
```

```
Attaching package: ‘dplyr’

The following objects are masked from ‘package:stats’:

    filter, lag

The following objects are masked from ‘package:base’:

    intersect, setdiff, setequal, union
```

Conjunto de dados do pipeline são passados pelo operador |> para função seguinte no primeiro parâmetro da função de modo implícito

Pipeline: Combining operations (filtering, projection, and sort)

```
result <- dsm |>
    filter(height>1.7) |>
    select(subject, weight, height) |>
    arrange(height)
head(result)
```

A data.frame: 5 × 3

	subject	weight	height
	<chr>	<dbl>	<dbl>
1	E	95	1.74
2	A	60	1.75
3	B	72	1.80
4	D	90	1.90
5	F	72	1.91

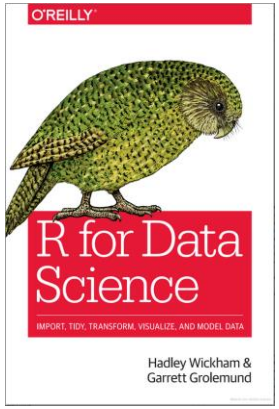
Pipeline (data aggregation)

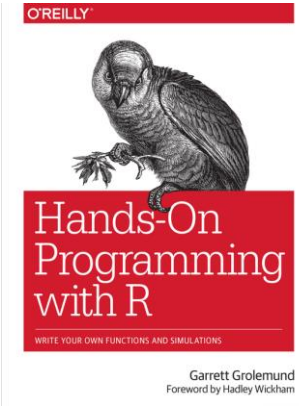
```
result <- dsm |>
    group_by(state) |>
    summarize(count = n(), height = mean(height))
head(result)
```

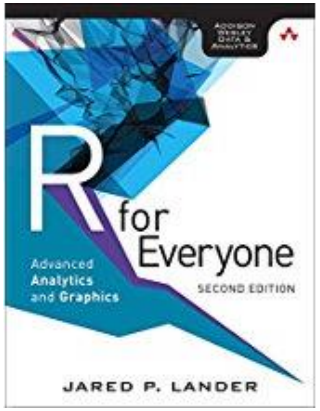
A tibble: 3 × 3

state	count	height
<chr>	<int>	<dbl>
MG	2	1.780
RJ	2	1.825
SP	2	1.770

Main References







Major reference for this class is R for everyone

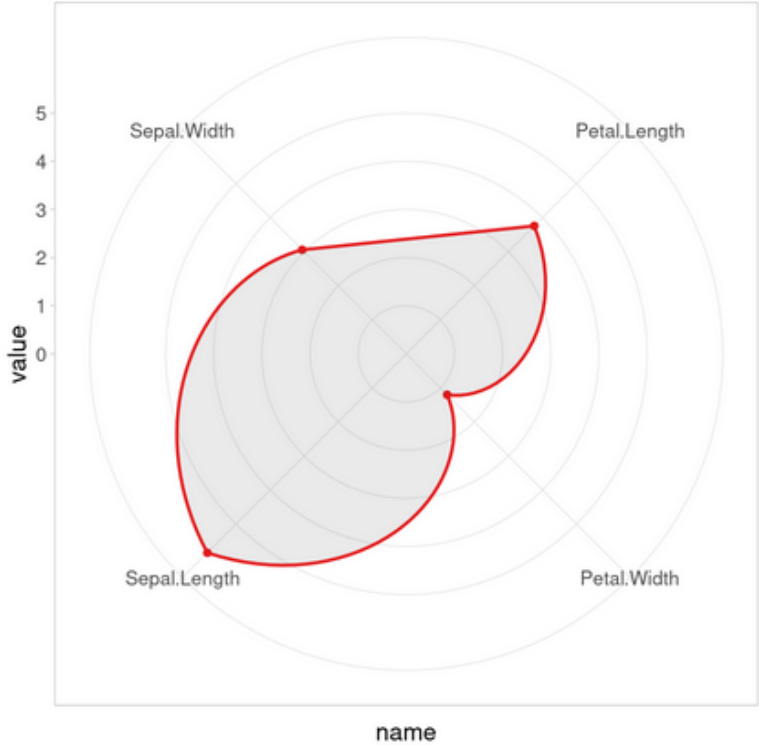
Additional Material

Radar plot

A radar chart is a graphical method of displaying multivariate data containing three or more quantitative variables represented on axes starting from the same point.

More information: https://en.wikipedia.org/wiki/Radar_chart

```
grf <- plot_radar(data, colors=colors[1]) + font
grf <- grf + ylim(0, NA)
plot(grf)
```



https://github.com/cefet-rj-dal/daltoolbox/blob/main/graphics/grf_radar.md

Statistical analysis

t-test

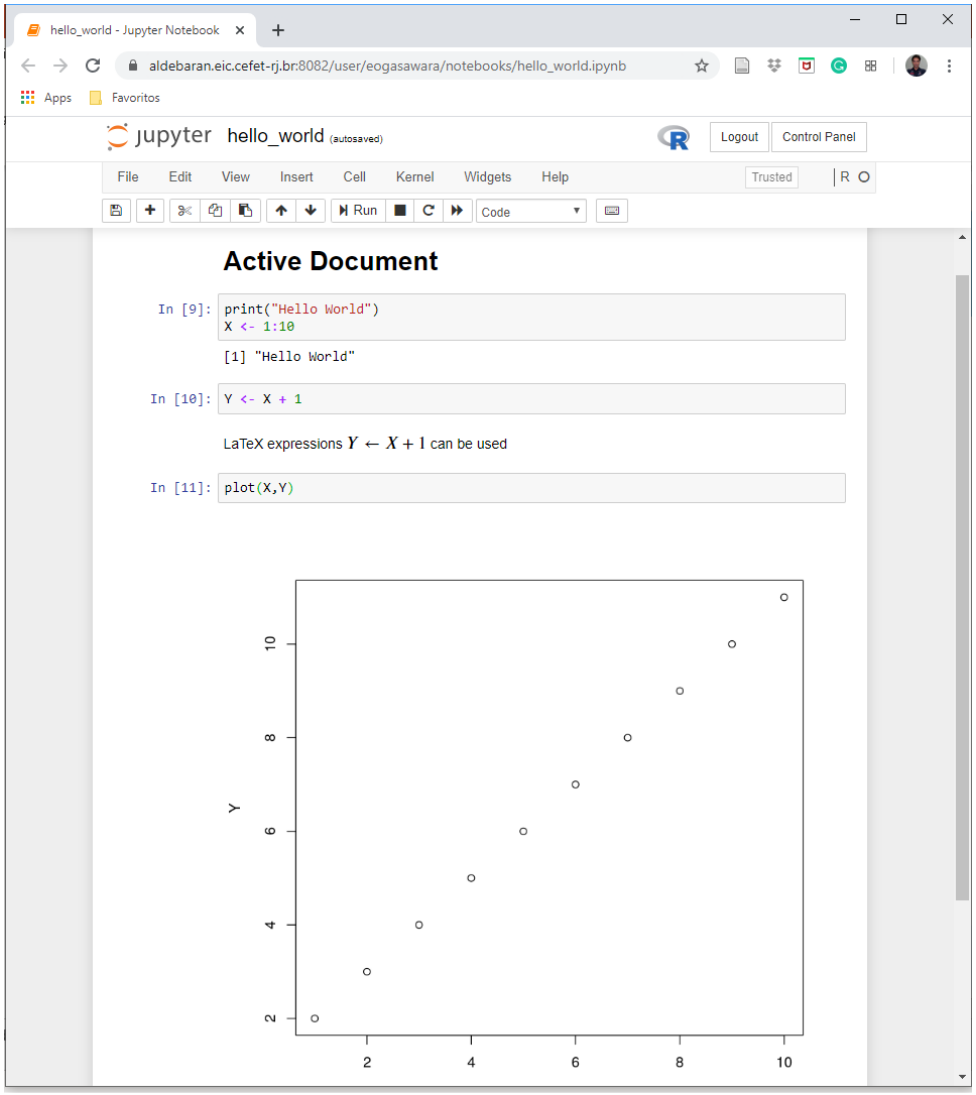
```
weight <- c(60, 72, 57, 90, 95, 72)
height <- c(1.75, 1.80, 1.65, 1.90, 1.74, 1.91)
bmi <- weight/height^2
```

```
t.test(bmi, mu=22.5)
```

```
        One Sample t-test

data:  bmi
t = 0.34488, df = 5, p-value = 0.7442
alternative hypothesis: true mean is not equal to 22.5
95 percent confidence interval:
 18.41734 27.84791
sample estimates:
mean of x
 23.13262
```

Test theoretical value of BMI equals to 22.5. Null hypothesis: no difference observed (p-value > 5%) Alternative hypothesis: they are different.
https://www.healthknowledge.org.uk/public-health-textbook/research-methods/1b-statistical-methods/parametric-nonparametric-tests

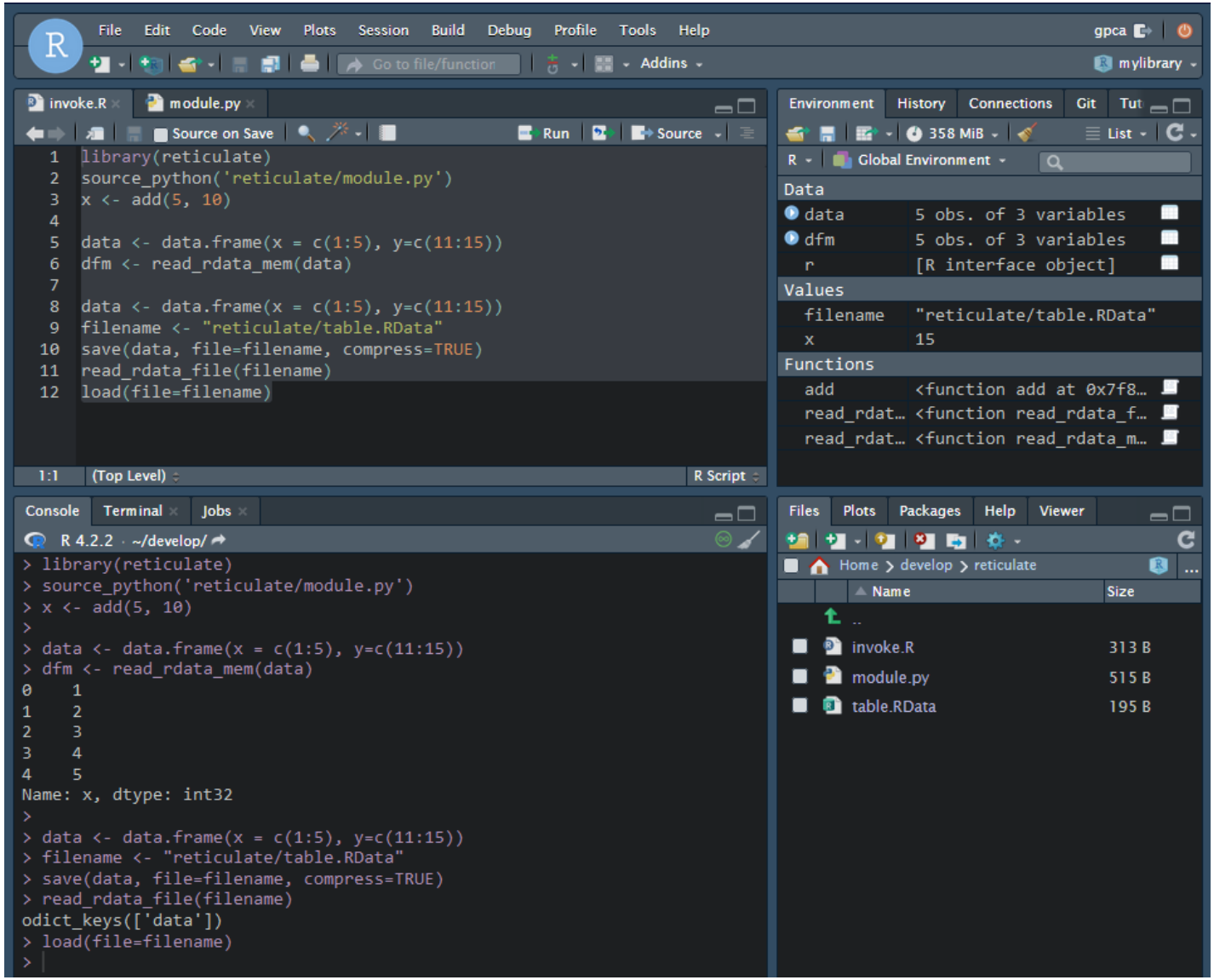
Jupyter Notebook



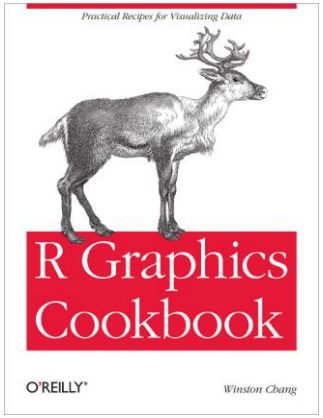
Python + R integration

```python
import pyreadr
import pandas

def add(x, y):
  return x + y

def read_rdata_mem(data):
  x = data["x"]
  print(x)
  y = data["y"]
  data["z"] = x + y
  return(data)

def read_rdata_file(filename):
  result = pyreadr.read_r(filename)
  print(result.keys()) # let's check what objects we got: there is only None
  df1 = result["data"] # extract the pandas data frame for the only object available
  x = df1["x"]
  y = df1["y"]
  df1["z"] = x + y
  pyreadr.write_rdata(filename, df1, df_name="data", compress="gzip")
```

Python + R integration



Main References



Further reading: Hands-On Programming with R, Chapters 1-12

Slides e vídeos em: https://eic.cefet-rj.br/~eogasawara/data-mining/

- https://rstudio-education.github.io/hopr/index.html
- https://r-graphics.org